

BUSN 5000 Project Guide

Chris Cornwell and Abbi Cormier

May 18, 2026

Table of contents

Preface	1
How to use this guide	1
Acknowledgments	1
1 Setup	3
1.1 Create your BUSN 5000 folder	3
1.2 Install R and RStudio	3
1.2.1 Windows	3
1.2.2 macOS	4
1.3 Install the required R packages	5
1.4 Create the subfolders inside Project/	5
1.5 Sort the Pre-project file pack into the right places	5
1.6 Create the .Rproj file	6
1.7 Run the setup check	7
1.8 Common setup mistakes	7
1.8.1 Mistake 1: project_slidedeck.css in the wrong folder	7
1.8.2 Mistake 2: Working from your Downloads folder	7
1.8.3 Mistake 3: Knitting before running the R script	7
1.9 What happens next	8
2 Pre-project	9
2.1 What you'll produce	9
2.2 Workflow	9
2.2.1 Step 1: Open the project in RStudio	9
2.2.2 Step 2: Run LF.R	9
2.2.3 Step 3: Open pre_project.Rmd	10
2.2.4 Step 4: Fill in the YAML author line	10
2.2.5 Step 5: Complete the code chunk on Slide 2	10
2.2.6 Step 6: Fill in the paragraph on Slide 3	11
2.2.7 Step 7: Knit the Rmd to HTML	11
2.2.8 Step 8: Compare your output to the reference	12
2.2.9 Step 9: Save the HTML as PDF	12
2.2.10 Step 10: Submit to Gradescope	12
2.3 Common pre-project mistakes	12
2.4 What happens next	13
3 Main Project	15
3.1 Outline (to be written)	15
4 Progress Check	17
4.1 Why bother	17
4.2 What you'll do	17
4.3 What's graded	17
4.4 Rubric	18

4.5	What's <i>not</i> graded	18
4.6	Submission	19
4.7	Common pitfalls	19
4.8	What happens next	19
5	Submission	21
5.1	Filename conventions	21
5.2	Late submission policy	21
5.3	The submission workflow at a glance	21
5.4	Saving HTML as PDF	22
5.4.1	Google Chrome (and Microsoft Edge)	22
5.4.2	Firefox	23
5.4.3	Safari	25
5.5	Pre-project reference: what your three slides should look like	26
5.5.1	Reference slide 1 — Title slide	27
5.5.2	Reference slide 2 — Read LF.csv and print its contents	28
5.5.3	Reference slide 3 — Document the contents of LF.csv	29
5.6	Final Project reference: what your 35 slides should look like	29
5.6.1	Front matter (slides 1–2)	30
5.6.2	Introduction (slides 3–4)	32
5.6.3	Data (slides 5–8)	34
5.6.4	Baseline earnings distributions (slides 9–12)	38
5.6.5	The career gender gap (slides 13–20)	42
5.6.6	Explaining the gender wage gap (slides 21–30)	50
5.6.7	Conclusion (slides 31–32)	60
5.6.8	Appendix (slides 33–35)	62
5.7	If your output doesn't match the reference	64
6	Common Errors and FAQ	65
6.1	Outline (to be written)	65
7	R and AI Resources	67
7.1	Resources for learning and using R	67
7.1.1	IDEs for R	67
7.1.2	Learning R	67
7.1.3	Workflow	67
7.2	AI resources	68
7.2.1	Working with AI	68
7.2.2	AI for coding	68
7.2.3	Learning more about AI	68

Preface

Welcome to the **BUSN 5000 Project Guide** — the canonical reference for the Pre-project, Project Progress Check, and Final Project for BUSN 5000 (Intro to Data Science for Business and Economics) at the University of Georgia.

This guide covers:

- How to set up your computer and your project folder
- What the pre-project is and how to complete it
- What the main project is and how to complete it
- What the progress check is and how it's graded
- How to submit your work
- How to recognize and recover from common errors

The guide is the single source of truth. Announcements on eLC remind you of upcoming deadlines and point back here for the rules.

How to use this guide

Read the chapters in order the first time through. After that, treat it as a reference — jump to whatever section you need. Each chapter has its own table of contents in the sidebar.

If you find a section that's wrong, unclear, or out of date, please flag it via the “report an issue” link at the top of any page.

Acknowledgments

The course materials in this guide are by **Chris Cornwell** and **Abbi Cormier**, with substantial contributions from the BUSN 5000 teaching team across many semesters.

1Setup

This chapter walks you through getting your computer ready to do BUSN 5000 project work. By the end of it, you should have:

- R and RStudio installed
- All required R packages installed
- A **Project/** folder on your computer with the correct subfolder structure
- Every file from the Pre-project file pack sorted into the right place
- An **.Rproj** file that opens RStudio into the project
- A green report from the setup-check script

Plan to spend 30–45 minutes the first time through.

! Don't skip this

Almost every “my project won't knit” email we get traces back to something here. If setup is right, the rest of the project is much easier. If setup is wrong, every other step compounds the problem.

1.1 Create your BUSN 5000 folder

Before you do anything project-specific, set up a top-level **BUSN 5000** folder on your computer. Put it somewhere you'll remember — your **Documents** folder is a fine default.

Inside **BUSN 5000**, create two subfolders:

```
BUSN 5000/  
  Homework/  
  Project/
```

- **Homework/** is where you'll save homework assignments and other course materials over the semester.
- **Project/** is the self-contained working directory for the Pre-project, Project Progress Check, and Final Project.

The rest of this chapter is about setting up **Project/**. We won't touch **Homework/** here.

1.2 Install R and RStudio

If you haven't already, install both. Follow the instructions for your operating system. Install R before RStudio.

1.2.1 Windows

Step 1: Install R

1. Go to the CRAN R Project website.
2. Click on “Download R for Windows”.
3. Click on “base” and then select the link labeled “Download R x.x.x for Windows” (where x.x.x is the latest version).
4. Open the downloaded `.exe` file and follow the installation prompts, accepting the defaults.

Step 2: Install RStudio

1. Visit the RStudio download page.
2. Click the link to download the Windows installer.
3. Open the downloaded `.exe` file and complete the installation using default settings.

Step 3: Verify the installation

1. Launch RStudio from the Start menu or desktop shortcut.
2. In the Console pane, type `print("Hello, world!")` and press Enter.
3. If you see `[1] "Hello, world!"`, your installation is successful.

1.2.2 macOS

First, identify your Mac’s chip. Newer Macs use Apple Silicon (M1, M2, M3, M4, etc.); older ones use Intel processors. You need to know which you have to pick the right R installer.

1. Click the Apple menu (top-left corner) → **About This Mac**.
2. Look at the **Chip** (Apple Silicon) or **Processor** (Intel) line:
 - If it says “Apple M1”, “M2”, “M3”, “M4”, or any Apple-labeled chip, you have **Apple Silicon**. You want the `arm64` installer.
 - If it says “Intel Core ...”, you have an **Intel Mac**. You want the `x86_64` installer.

Step 1: Install R

1. Visit the CRAN R Project website.
2. Click on “Download R for macOS”.
3. Select the latest version matching your chip — either `R-x.x.x-arm64.pkg` (Apple Silicon) or `R-x.x.x-x86_64.pkg` (Intel).
4. Open the downloaded `.pkg` file and follow the installation prompts.

Step 2: Install RStudio

1. Go to the RStudio download page.
2. Click the link to download the macOS version of RStudio.
3. Open the downloaded `.dmg` file and drag the RStudio icon to your **Applications** folder.

Step 3: Verify the installation

1. Open RStudio from your **Applications** folder.
2. In the Console pane, type `print("Hello, world!")` and press Return.
3. If you see `[1] "Hello, world!"`, your installation is successful.

1.3 Install the required R packages

Install all seven packages using the **Packages** pane in RStudio:

1. Select the **Packages** tab in the bottom-right pane.
2. Click **Install** at the top of that pane.
3. In the “Install Packages” box, enter the package names separated by spaces or commas:
here tidyverse modelsummary ggpmisc car ggrepel kableExtra
4. Make sure **Install dependencies** is checked.
5. Click **Install**.

Installation takes a few minutes the first time — let it finish completely.

Tip

You only install packages **once per machine**. After that, the templates load them with `library()` calls inside the Rmd setup chunks, which run every time you knit. Don't put `install.packages()` calls inside an `.Rmd` — that would re-install every time you knit.

1.4 Create the subfolders inside Project/

Inside your BUSN 5000/Project/ folder, create these four subfolders:

- data/
- out/
- r/
- css/

So your skeleton looks like:

```
BUSN 5000/  
  Homework/  
  Project/  
    data/  
    out/  
    r/  
    css/
```

You'll put files into these subfolders in the next step. The `.Rmd` files themselves will live in the `Project/` root, **not** in any of the subfolders.

1.5 Sort the Pre-project file pack into the right places

When you download the Pre-project file pack from eLC, you'll get a flat bucket of 7 files. Your job is to figure out where each one belongs in your `Project/` directory. This isn't busywork — knowing

which file goes where is itself part of what we want you to learn.

Here's the destination for each file in the pack:

File	Goes in
pre_project.Rmd	Project/ (root)
preproject_instructions.pdf	Project/ (root)
LF.R	Project/r/
project_slidedeck.css	Project/css/
pppub24.csv	Project/data/
hhpub24.csv	Project/data/
check_setup_preproject.R	Project/r/

After sorting, your folder should look like:

```
Project/
  pre_project.Rmd
  preproject_instructions.pdf
  data/
    pppub24.csv
    hhpublish24.csv
  out/
    (empty for now - LF.R will write LF.csv here)
  r/
    LF.R
    check_setup_preproject.R
  css/
    project_slidedeck.css
```

1.6 Create the .Rproj file

The `.Rproj` file is what tells RStudio “this folder is an R project.” It anchors all the relative paths the templates use.

To create one:

1. Open RStudio.
2. **File** → **New Project**.
3. Choose **Existing Directory**.
4. Browse to your `Project/` folder. Click **Create Project**.
5. RStudio re-opens with `Project` loaded as the active project.

You'll see `Project.Rproj` appear in the `Project/` root. From now on, **always open RStudio by double-clicking that `.Rproj` file**. That guarantees your working directory and paths are correct.

1.7 Run the setup check

1. In RStudio's **Files** pane (bottom-right), open the `r/` subfolder.
2. Click `check_setup_preproject.R` to open it in the editor.
3. In the editor, **Select All** (Ctrl+A on Windows, Cmd+A on Mac).
4. Click the **Run** button in the top-right of the editor pane (or press Ctrl+Enter / Cmd+Enter).

The script's output appears in the Console pane. You'll see one line per file: either (good) or with a specific instruction telling you what to fix.

Note

If you see any , fix the listed problems and re-run the script. Don't move past Setup until every line is a .

1.8 Common setup mistakes

In our experience, these are the three setup problems we see most often. Each one is preventable with a few seconds of attention.

1.8.1 Mistake 1: `project_slidedeck.css` in the wrong folder

The CSS file styles your slide deck. If it's anywhere but `Project/css/project_slidedeck.css`, your knitted slides will lose all of their formatting and the submission will be rejected. The setup-check script will catch this, but it's a simple thing to get right on the first pass.

1.8.2 Mistake 2: Working from your Downloads folder

If you double-click `pre_project.Rmd` directly from your Downloads folder, RStudio's working directory is Downloads, not your `Project/` folder. The `here()` paths in the `.Rmd` will resolve to the wrong place and you'll get cryptic "file not found" errors.

Always move the files into your `Project/` folder before opening them. Your Downloads folder is a transit area, not a workspace.

1.8.3 Mistake 3: Knitting before running the R script

The `.Rmd` reads output that the `LF.R` script (and later `cpsmar_e.R`) produces. If you haven't run the script yet, the output file doesn't exist, and the `.Rmd` errors out the moment it tries to read `out/LF.csv`.

Workflow order for each stage:

1. Open the `.Rproj`.
2. Run the relevant R script (`LF.R` for pre-project, `cpsmar_e.R` for main project).
3. *Then* open the `.Rmd` and knit it.

The setup-check script can confirm files exist, but only running the script confirms it actually produced the output the `.Rmd` needs.

1.9 What happens next

You're set up. From here:

- For the **Pre-project**, continue to Chapter 3.
- For the **Main Project** (later in the semester), you'll add more files to the same **Project/** folder — see Chapter 4.
- For the **Project Progress Check**, you continue working on the same main project files — see Chapter 5.
- For supplementary materials on R, RStudio, AI tools, and coding practices, see R and AI Resources.

Your **Project/** directory carries through the entire semester. You won't need to re-do Setup unless you're working from a new machine.

2Pre-project

The Pre-project Exercise is a short, required exercise designed to get you and your computer ready for the Main Project. By completing it correctly, you'll have:

- Installed the required R packages
- Set up the proper directory structure on your computer
- Learned how to knit your `.Rmd` to an HTML slide deck and save it in PDF format
- Earned the **5% of your Project grade** that the Pre-project is worth

! All-or-nothing

The Pre-project is worth **5% of your Project grade**, graded all-or-nothing. If your content is correct **and** your slide deck is 100% compliant with the formatting in `pre_project_knitted_template.html`, you earn the full 5%. Any deviation — content error, formatting error, wrong filename, anything — and you earn 0. There is no partial credit on the Pre-project.

This chapter assumes you've completed Setup. If you haven't created your `Project/` folder, sorted the file pack, created the `.Rproj`, and run `check_setup_preproject.R`, go do that first.

2.1 What you'll produce

A 3-slide PDF slide deck:

1. **Title slide** — auto-generated from the YAML block; just requires your name.
2. **Read `LF.csv` and print its contents** — one code chunk you complete.
3. **Document the contents of `LF.csv`** — one paragraph with six blanks to fill in.

That's it. Three slides.

2.2 Workflow

The order matters. Follow these steps in sequence.

2.2.1 Step 1: Open the project in RStudio

Double-click `Project.Rproj` in your `BUSN 5000/Project/` folder. RStudio opens with the project loaded. You should see "Project" in the top-right corner of the RStudio window.

2.2.2 Step 2: Run `LF.R`

`LF.R` reads the raw CPS data file (`pppub24.csv`), computes the labor force participation rate and its components, and writes the results to `out/LF.csv`. You need to run it before you can knit the `Rmd`.

1. In RStudio's **Files** pane (bottom-right), open the `r/` subfolder.
2. Click `LF.R` to open it in the editor.
3. **Select All** (Ctrl+A on Windows, Cmd+A on Mac).
4. Click the **Run** button (or press Ctrl+Enter / Cmd+Enter).

The console will print a message like `March 2024 LFPR = 62.7...`. If you see it, the script ran. A new file `LF.csv` now lives in your `out/` subfolder.

! Important

You must run `LF.R` **before** you knit `pre_project.Rmd`. The Rmd reads `out/LF.csv`; if that file doesn't exist, the Rmd errors out.

2.2.3 Step 3: Open `pre_project.Rmd`

In the Files pane, click `pre_project.Rmd` (it lives in the `Project/` root, not in any subfolder). It opens in the editor.

2.2.4 Step 4: Fill in the YAML author line

Near the top of the file, find the YAML block (the section between the `---` markers). Look for this line:

```
author: "firstname lastname"
```

Replace `firstname lastname` with your **first and last name**, e.g.:

```
author: "Jane Doe"
```

Don't change anything else in the YAML. This is the most common place students introduce formatting errors.

2.2.5 Step 5: Complete the code chunk on Slide 2

Slide 2 is titled "Read `LF.csv` and print its contents." The code chunk looks like:

```
lf <- read_csv(here("out", "LF.csv"))
print(__)
```

If you've never seen R code before, here's what's happening line by line:

- **Line 1.** `lf <- read_csv(here("out", "LF.csv"))` does three things at once. `read_csv("...")` is a function that reads a CSV file. `here("out", "LF.csv")` builds the file path `out/LF.csv` relative to your project root — this is why you ran `LF.R` first; it created the `LF.csv` that this line is reading. And `lf <- ...` takes the result and stores it in a new **object** named `lf`. The arrow `<-` is R's assignment operator (you'll see it used the same way most other languages use `=`).

- **Line 2.** `print(...)` prints whatever you put inside the parentheses to the slide output. The blank `__` is where you tell R *what* to print.

Replace `__` with the object you want to print. Since you just read the contents into `lf`, the answer is `lf`:

```
lf <- read_csv(here("out", "LF.csv"))
print(lf)
```

Then change the chunk option `eval = FALSE` to `eval = TRUE` at the top of the chunk. This tells R Markdown to actually run the chunk when you knit (it's set to `FALSE` by default so the template knits before you complete it).

Run the chunk (click the small green play button at the top-right of the chunk) to verify it works. You should see the `lf` data frame printed in the console — six columns named `E`, `U`, `LF`, `NILF`, `Pop`, and `LFPR`, with one row of numbers.

Write down those six numbers — you'll use them in the next step.

2.2.6 Step 6: Fill in the paragraph on Slide 3

Slide 3 is titled “Document the contents of `LF.csv`.” It has a paragraph with six blanks (each blank is shown as `_____`). The paragraph reads:

The `LF.csv` file contains the results of the labor force participation rate (LFPR) calculation using the March 2024 CPS produced by the script, `LF.R`. The LFPR (_____) is the ratio of the labor force (_____) to the population (____). The labor force is the sum of the employed (_____) and unemployed (____). The population is the sum of the labor force and those not in the labor force (____). The LFPR is expressed as a percentage.

Each blank goes inside parentheses that follow a specific term — fill in the value of whatever term is named directly before the parentheses, rounded to two decimal places. For example, the first blank follows “the LFPR”, so it gets the LFPR value; the second follows “the labor force”, so it gets the labor force value; and so on.

For reference, here's the mapping from each blank to the matching column in your `lf` data frame:

Blank	Value from <code>lf</code>
LFPR	LFPR
labor force	LF
population	Pop
employed	E
unemployed	U
not in labor force	NILF

2.2.7 Step 7: Knit the Rmd to HTML

With your changes saved, click the **Knit** button at the top of the editor pane.

The knit produces an HTML file in your `Project/` folder. It opens automatically in the RStudio Viewer or your default browser.

2.2.8 Step 8: Compare your output to the reference

The Submission chapter includes a PDF showing exactly what each of the three pre-project slides should look like when rendered correctly — same fonts, same layout, same code chunk styling. Open it side-by-side with your knitted output and confirm they match.

If your output doesn't match exactly, something is wrong. Common causes are listed in the Common Errors chapter.

2.2.9 Step 9: Save the HTML as PDF

You don't knit-to-PDF directly. Instead, you save the HTML output as a PDF using your browser's Save As / Print to PDF function. See the Submission chapter for the exact procedure and browser-specific tips.

2.2.10 Step 10: Submit to Gradescope

Submission filename: `firstname_lastname_pre.pdf` (e.g., `jane_doe_pre.pdf`).

Upload to the **Preproject 0 Exercise** assignment in Gradescope.

2.3 Common pre-project mistakes

In addition to the Setup-stage mistakes in Common Errors (CSS in the wrong folder, working from Downloads, knitting before running the R script), these pre-project-specific mistakes are the ones we see most:

1. **Leaving `eval = FALSE` after completing the code chunk.** If `eval = FALSE`, the chunk doesn't run when you knit, so the output doesn't appear on the slide. Set it to `TRUE` once your code is correct.
2. **Numbers in the paragraph don't match `LF.csv`.** This usually means a student copied numbers from a peer instead of running `LF.R` themselves. The numbers you fill in must come from your own `out/LF.csv` after running your own copy of `LF.R`.
3. **YAML edits beyond the author line.** Changing anything in the YAML besides the author name will break the knit format. Don't touch the `title`, `subtitle`, `date`, `output`, `css`, or any other field.
4. **CSS not correctly placed.** If `project_slidedeck.css` is anywhere other than `Project/css/`, the knit produces an unstyled output that won't match the template. See Mistake 1 in Common Errors.
5. **Saving the wrong way.** The submission must be a PDF generated by saving the *HTML* output, not by knitting directly to PDF. See Submission for the correct workflow.

When in doubt, see Common Errors — most issues have already been documented.

2.4 What happens next

Once you've submitted the Pre-project, you've done the hardest part: your environment works. The Main Project will reuse the same **Project/** folder and the same workflow patterns. Continue to Chapter 4: Main Project when you're ready.

3Main Project

This chapter is the slide-by-slide guide to the BUSN 5000 Project: **Exploring Pay Differences between Women and Men**.

3.1 Outline (to be written)

- Project overview and learning goals
- Data: March 2024 CPS ASEC
- The 35-slide deck structure
- Slide-by-slide instructions (migrated from the current Instructions HTML)
- Point allocations per slide
- Line limits and write-up rules

Note

This chapter is a stub. Content will be filled in as part of Wave 2 of the project rebuild.

4Progress Check

The Project Progress Check is an **optional** mid-project submission that gives you a small bonus on your Project grade in exchange for showing that the core tables and figures are on track. It's offered in Week 4 of the semester, halfway between the Pre-project and the Final Project.

You don't have to submit it. We strongly recommend that you do.

4.1 Why bother

Two reasons:

1. **Bonus points.** A complete and correct progress check earns up to **5 bonus points** on your Project grade (per the syllabus).
2. **A safety net for the Pre-project.** If you earned a 0 on the Pre-project, the progress check is your one opportunity to earn those points back. Submit a clean progress check and the lost pre-project points are recovered.

Even if you passed the Pre-project, treating the progress check as required is the safer move — it forces you to confirm your formatting is right and your core figures are working well before the final deadline.

4.2 What you'll do

The progress check is not a separate file or a separate document. You continue working on the same `project.Rmd` you started after finishing the Pre-project. On the progress check deadline, you save the in-progress `.Rmd` as a PDF (using the same workflow as the Pre-project and Final Project — see Submission) and upload it to the Project Progress Check assignment on Gradescope.

There is no “progress check version” of the project. There's just the project, as it stands at the progress-check deadline.

4.3 What's graded

The progress check covers **about two-thirds of the project's core tables and figures**. Specifically, we look at six slides:

Slide	Title	What we check
1	Title slide	Your name filled in (and overall formatting is correct)
2	Academic Honesty Statement	Your name on the Signature line
11	Distribution of earnings by gender	Figure 1 is visible, completed, and correct

Slide	Title	What we check
14	Wages and hours differences	Table 1 is visible, completed, and correct
17	Career log wage profiles	Figure 2 is visible, completed, and correct
19	Evolution of the gender wage gap	Table 2 is visible, completed, and correct

Note

The slide numbers above match the canonical 35-slide layout documented in the Final Project reference in the Submission chapter. Your deck must be 35 slides total even if many of the content slides are still placeholder.

4.4 Rubric

Three possible outcomes:

Result	Score	Criteria
Full credit	5 bonus points	All six graded slides are visible, completed, and correct, AND the deck is 100% compliant with the formatting requirements.
Partial credit	3 bonus points	The deck is 100% compliant with formatting, but at least one graded slide has a content error.
No credit	0 bonus points	Any formatting error (wrong CSS, wrong orientation, knit-to-PDF instead of HTML→PDF, etc.).

Important

Formatting errors are a hard zero. A perfectly correct content submission that's saved with the wrong orientation, lost CSS, or the wrong PDF workflow earns 0. Formatting first, then content.

4.5 What's *not* graded

- **Written sections.** We do not read the prose write-ups (Overview, Baseline comparisons, Documenting the differences, etc.) during the progress check. Those are evaluated only in

the Final Project submission.

- **Other tables and figures.** Slides 23, 24, 25, 29 (Table 3, Table 4, Figure 3, Table 5) are not part of the progress check. They're evaluated in the Final Project.

This focus is intentional — the progress check is a checkpoint, not a full grade. Use the un-graded slides for trying things out, sketching write-ups, etc.

4.6 Submission

- **Filename:** `firstname_lastname_progress.pdf` (e.g., `jane_doe_progress.pdf`).
- **Format:** 35-slide PDF, landscape, formatting compliant with the knitted reference in the Submission chapter.
- **Deadline:** As listed on the course schedule. Late submissions are not accepted.
- **Gradescope assignment:** “Project Progress Check 0”.

The save-as-PDF workflow is identical to the Pre-project and Final Project. See Submission for browser-by-browser print dialog settings.

4.7 Common pitfalls

These are the recurring failure modes from past semesters:

1. **Touching the setup chunk.** Do not change `include = FALSE` to `include = TRUE` on the setup chunk. The FALSE/TRUE toggle only applies to editing the `eval =` argument in the *content* code chunks (`bt11`, `bt12`, `mi1`, etc.), never the setup chunk.
2. **Skipping the extract script.** You must run `cpsmar_e.R` (and have the pre-project data files carried over from the Pre-project) before knitting. The `.Rmd` reads the extract it produces.
3. **Variable name confusion.** A common error: you save the analysis sample as `cpsmar_a` but then reference `cpsmar_e` in a later chunk (or vice versa). Read your code carefully and make sure each chunk references the right data frame.
4. **CSS in the wrong folder.** Same as the Pre-project — `project_slidedeck.css` must be in `Project/css/`. See Common Errors for diagnosis.
5. **Saving as PDF the wrong way.** Same as the Pre-project — you must knit to HTML first, then save the HTML as a PDF via your browser's print dialog (landscape orientation, background graphics enabled). Knitting directly to PDF strips the CSS formatting and earns a 0. See Submission for the per-browser walkthroughs.

4.8 What happens next

After the progress check, you continue working on `project.Rmd` toward the Final Project deadline. The work you did for the progress check stays in place — you fill in the remaining tables, figures, and write-ups, then submit the complete deck as `firstname_lastname_project.pdf`. See Chapter 4: Main Project for the slide-by-slide walkthrough and Submission for the final submission mechanics.

5Submission

This chapter covers how to submit your work to Gradescope without losing points to formatting. The same workflow applies to all three project artifacts you submit: the **Pre-project**, the optional **Project Progress Check**, and the **Final Project**.

In the past four years, the vast majority of pre-projects, project progress checks, and final projects that earned a 0 were the result of something done wrong here.

5.1 Filename conventions

Each of the three project artifacts has a required filename. Use **lowercase** for first and last name, **underscores** between words, and the **.pdf** extension.

Artifact	Filename pattern	Example
Pre-project	firstname_lastname_pre.pdf	jane_doe_pre.pdf
Project Progress Check	firstname_lastname_progress.pdf	jane_doe_progress.pdf
Final Project	firstname_lastname_project.pdf	jane_doe_project.pdf

! Important

Use the filename pattern exactly as written. It keeps your three submissions distinct in your project folder (so a later one doesn't overwrite an earlier one), and it makes each artifact easy to locate when you need to refer back to it. Required, not optional.

5.2 Late submission policy

Per the syllabus: **late submissions will not be accepted** for the Pre-project, the Project Progress Check, or the Final Project. The relevant deadlines are on the course schedule. There are no extensions and no late-credit options.

5.3 The submission workflow at a glance

Each submission goes through the same three steps:

1. **Knit** your `.Rmd` to HTML inside RStudio (the **Knit** button in the editor pane).

! Important

Click the **Knit** button itself, not the small dropdown arrow next to it. The dropdown offers “Knit to HTML / PDF / Word” options that override the output format we’ve configured in the Rmd’s YAML — picking one will overwrite our formatting and your

output will be rejected.

2. **Save** the resulting HTML page as a PDF using your browser's print-to-PDF function (instructions per browser below).
3. **Upload** the PDF to the corresponding Gradescope assignment under the correct filename.

 Do not knit directly to PDF

Knitting straight to PDF from R Markdown uses a different rendering path (LaTeX) that strips the CSS-based slide formatting we use. The output will not match the reference and **will receive a 0%** as a formatting violation.

You **must** knit to HTML, then save the HTML as PDF.

5.4 Saving HTML as PDF

After knitting, the HTML output opens in your browser (or in RStudio's Viewer pane — if it opens there, click the small icon at the top of the Viewer to open it in your default browser).

In every browser, the print dialog is what you use to save as PDF. The settings you need are roughly the same across browsers:

- **Destination:** “Save as PDF”
- **Layout / Orientation:** **Landscape**
- **Color:** Color (not Black and White)
- **Margins:** Default or None
- **Pages per sheet:** 1
- **Background graphics:** **Enabled** (this is the easy-to-miss one — without it, the colored UGA styling disappears)

Per-browser walkthroughs below.

5.4.1 Google Chrome (and Microsoft Edge)

Edge uses the same print dialog as Chrome — these steps work for both.

1. Click the three vertical dots in the top-right of the browser.
2. Click **Print**.
3. Match the settings shown below — most importantly, **Destination = Save as PDF**, **Layout = Landscape**, and **More settings → Background graphics = checked**.

Google Chrome (continued)

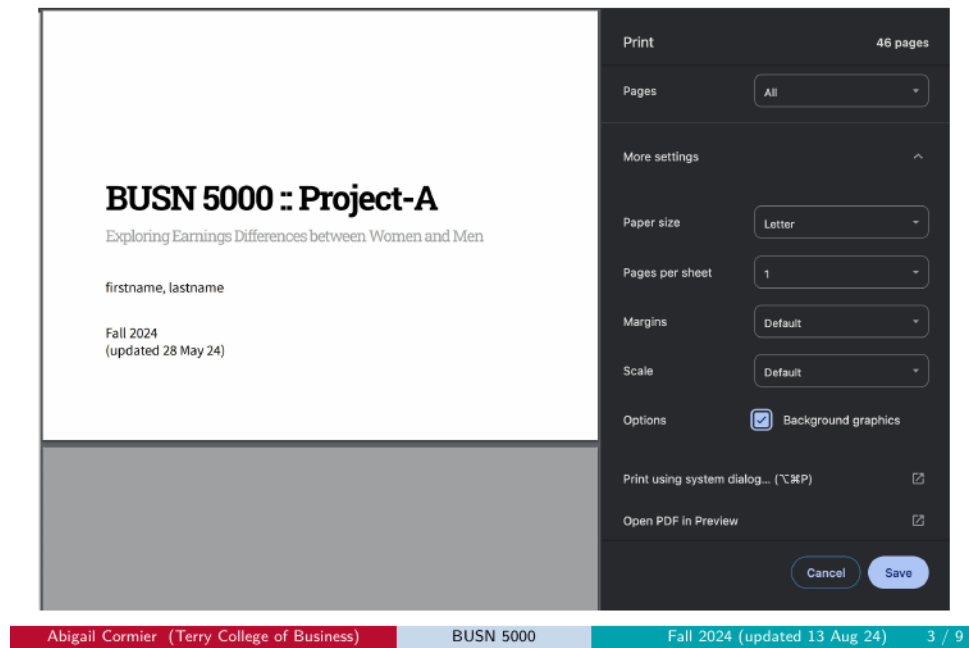


Figure 5.1: Chrome print dialog settings

4. Click **Save**. Use the correct filename when prompted (see Filename conventions).

5.4.2 Firefox

1. Click the three horizontal lines (the “hamburger” menu) in the top-right of the browser.
2. Click **Print**.
3. Match the settings shown below across the two screens. Key settings: **Destination = Save to PDF**, **Orientation = Landscape**, and **Options → Print backgrounds = checked**.

Firefox (continued)

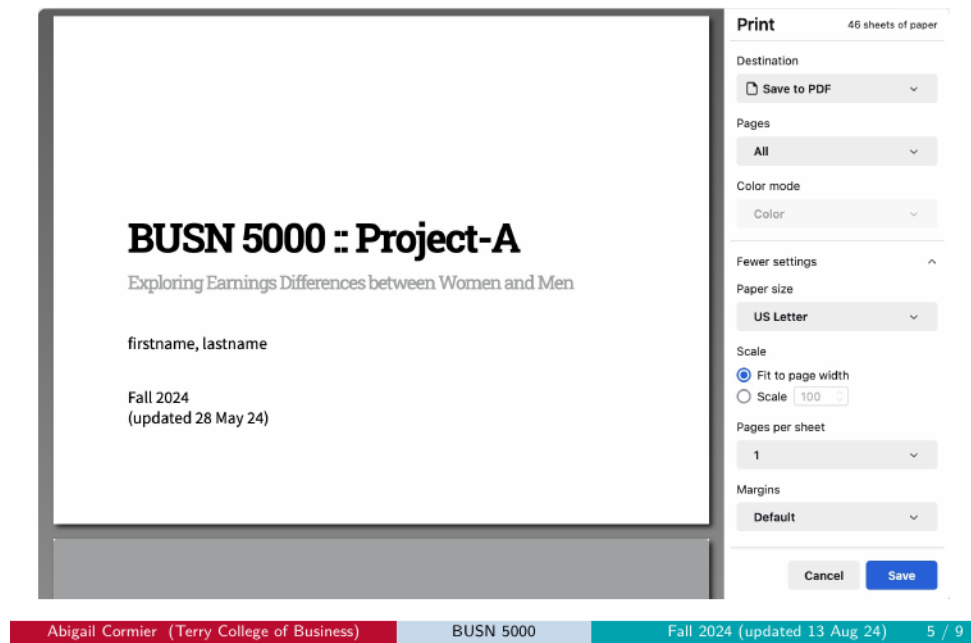


Figure 5.2: Firefox print settings — page 1

Firefox (continued)

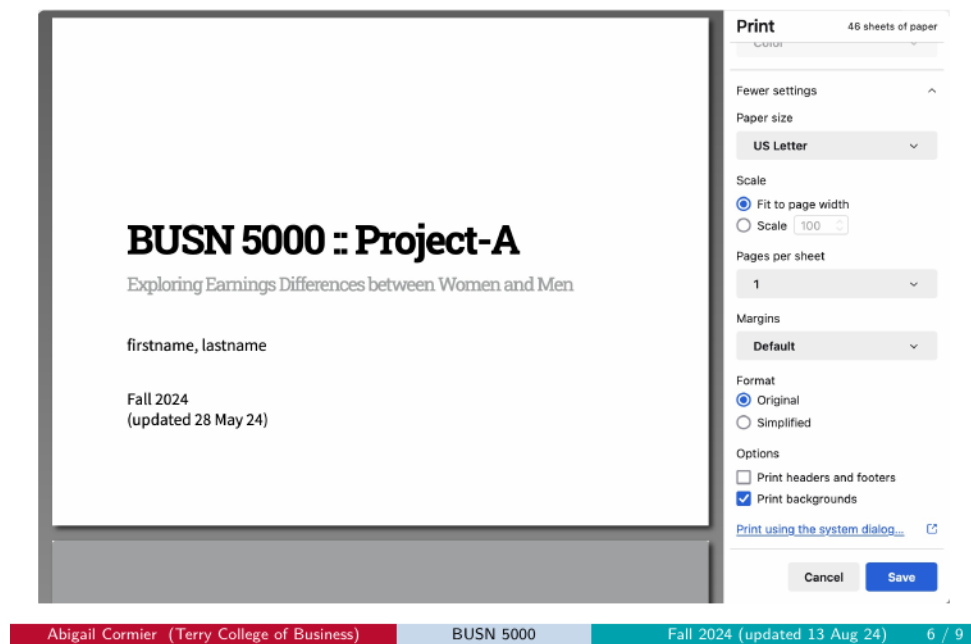


Figure 5.3: Firefox print settings — page 2

4. Click **Save** and use the correct filename.

5.4.3 Safari

1. Click **File** in the top-left menu bar.
2. Click **Print**.
3. Match the settings shown below across the two screens. Key settings: **Orientation = Landscape**, **Print backgrounds = checked**, and the **PDF dropdown in the bottom-left** must be set to **Save as PDF** (not Print).

Safari (continued)

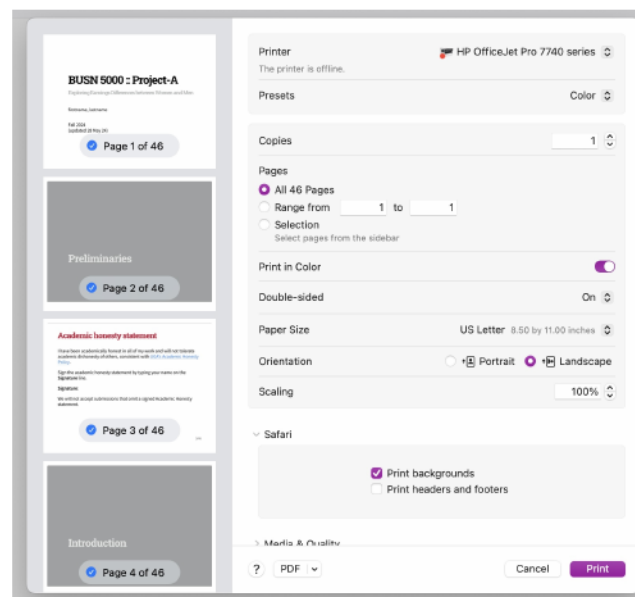


Figure 5.4: Safari print settings — page 1

Safari (continued)

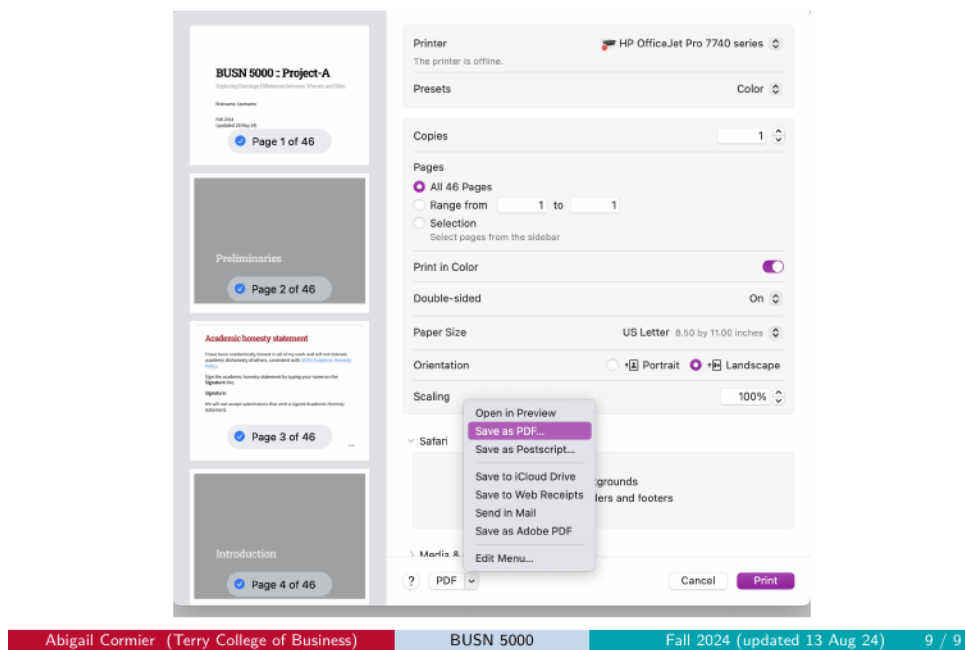


Figure 5.5: Safari print settings — page 2

4. Click **Save** and use the correct filename.

5.5 Pre-project reference: what your three slides should look like

Below are the three pre-project slides rendered correctly. Your knitted PDF should match these for **formatting** (fonts, colors, slide layout, code chunk styling). The **content** will differ — your name will be on the title slide and the numbers in your print-out and your fill-ins will reflect your `LF.csv`.

i Note

The images below show legacy examples (date stamp from a prior semester, comma in the name placeholder). They are reference for the slide *format*, not the exact text content. Your version will show Summer 2026, your name without a comma, and the current CPS numbers.

5.5.1 Reference slide 1 — Title slide

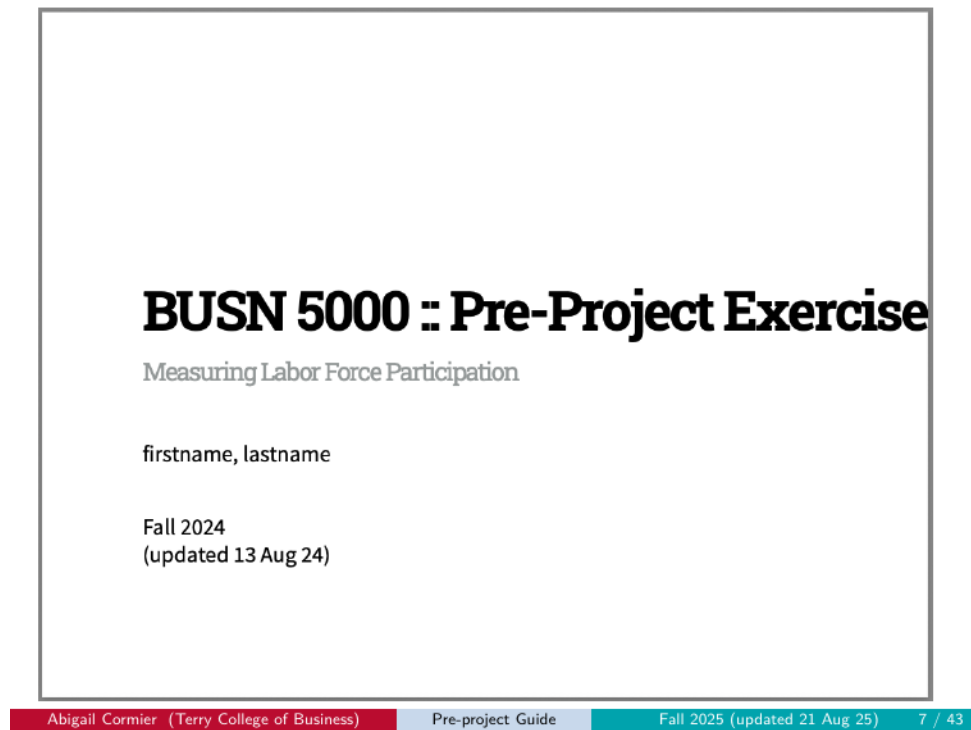
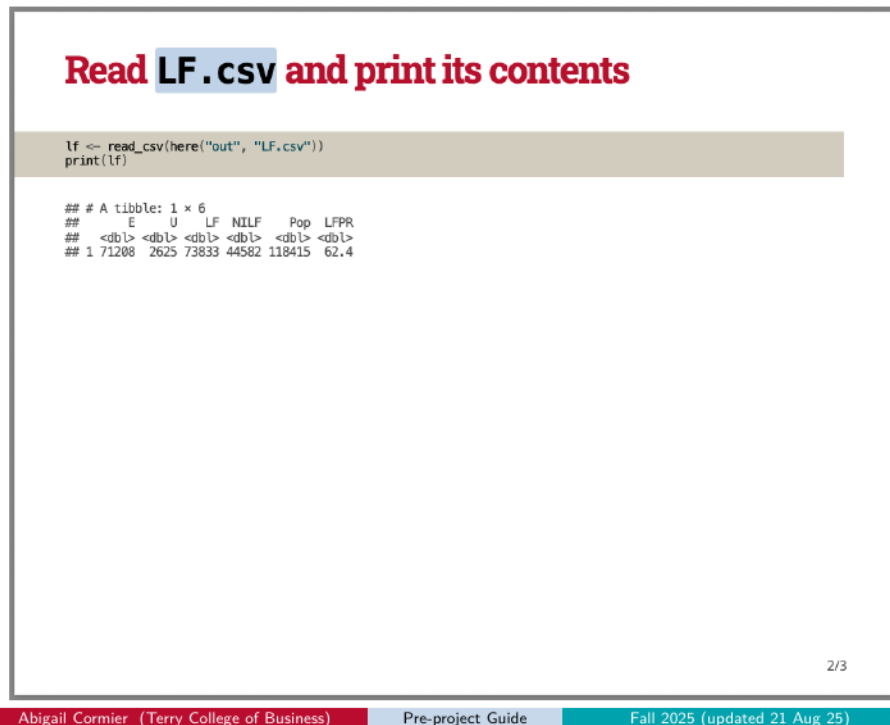


Figure 5.6: Title slide reference

What yours should look like:

- Title and subtitle as shown
- Your first and last name on the author line
- Summer 2026 date

5.5.2 Reference slide 2 — Read LF.csv and print its contents



Read LF.csv and print its contents

```
lf <- read_csv(here("out", "LF.csv"))
print(lf)
```

```
## # A tibble: 1 × 6
##       E      U    LF  NILF   Pop  LFPR
##   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1  71208  2625  73833  44582 118415  62.4
```

2/3

Abigail Cormier (Terry College of Business)

Pre-project Guide

Fall 2025 (updated 21 Aug 25)

9 / 43

Figure 5.7: LF print-out slide reference

What yours should look like:

- The code chunk visible (formatted with the UGA code-block style)
- The `lf` data frame printed below it with six named columns

5.5.3 Reference slide 3 — Document the contents of LF.csv

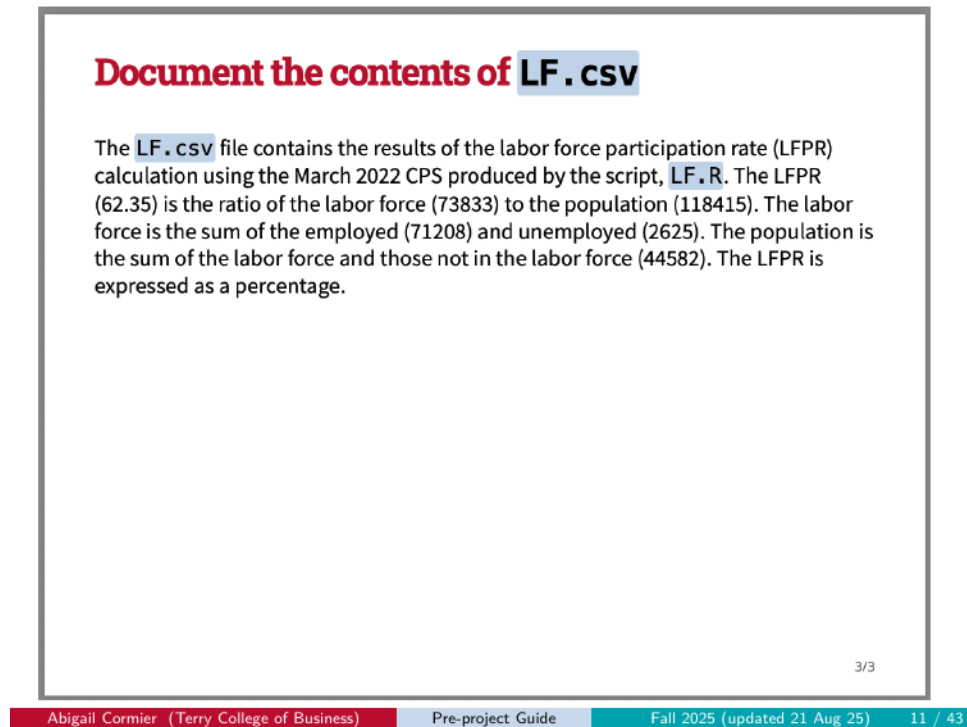


Figure 5.8: Communication slide reference

What yours should look like:

- The paragraph fully filled in, no remaining _____ blanks
- Values rounded to two decimal places
- Numbers matching your `out/LF.csv` (not numbers copied from this reference)

5.6 Final Project reference: what your 35 slides should look like

Below are the 35 final-project slides rendered correctly. Your knitted PDF should match these for **formatting** (fonts, colors, slide layout, tables, figures). The **content** will reflect your own work — your name on the title slide, your written sections, your data outputs, and your figures.

i Note

The images below show legacy examples (date stamp from a prior semester). The format is what to match. Your version will show Summer 2026 and current CPS numbers.

5.6.1 Front matter (slides 1–2)

BUSN 5000 Project

Exploring Pay Differences between Women and Men

First name Last name

Spring 2026
(updated 09 Jan 26)

Figure 5.9: Slide 1: Title

Academic honesty statement

I have been academically honest in all of my work and will not tolerate academic dishonesty of others, consistent with [UGA's Academic Honesty Policy](#).

Sign the academic honesty statement by typing your name on the **Signature** line.

Signature:

We will not accept submissions that omit a signed Academic Honesty statement.

2/35

Figure 5.10: Slide 2: Academic Honesty Statement

5.6.2 Introduction (slides 3–4)



Figure 5.11: Slide 3: Introduction (section divider)

Overview

Insert your content here replacing these words with your own text.

4/35

Figure 5.12: Slide 4: Overview

5.6.3 Data (slides 5–8)

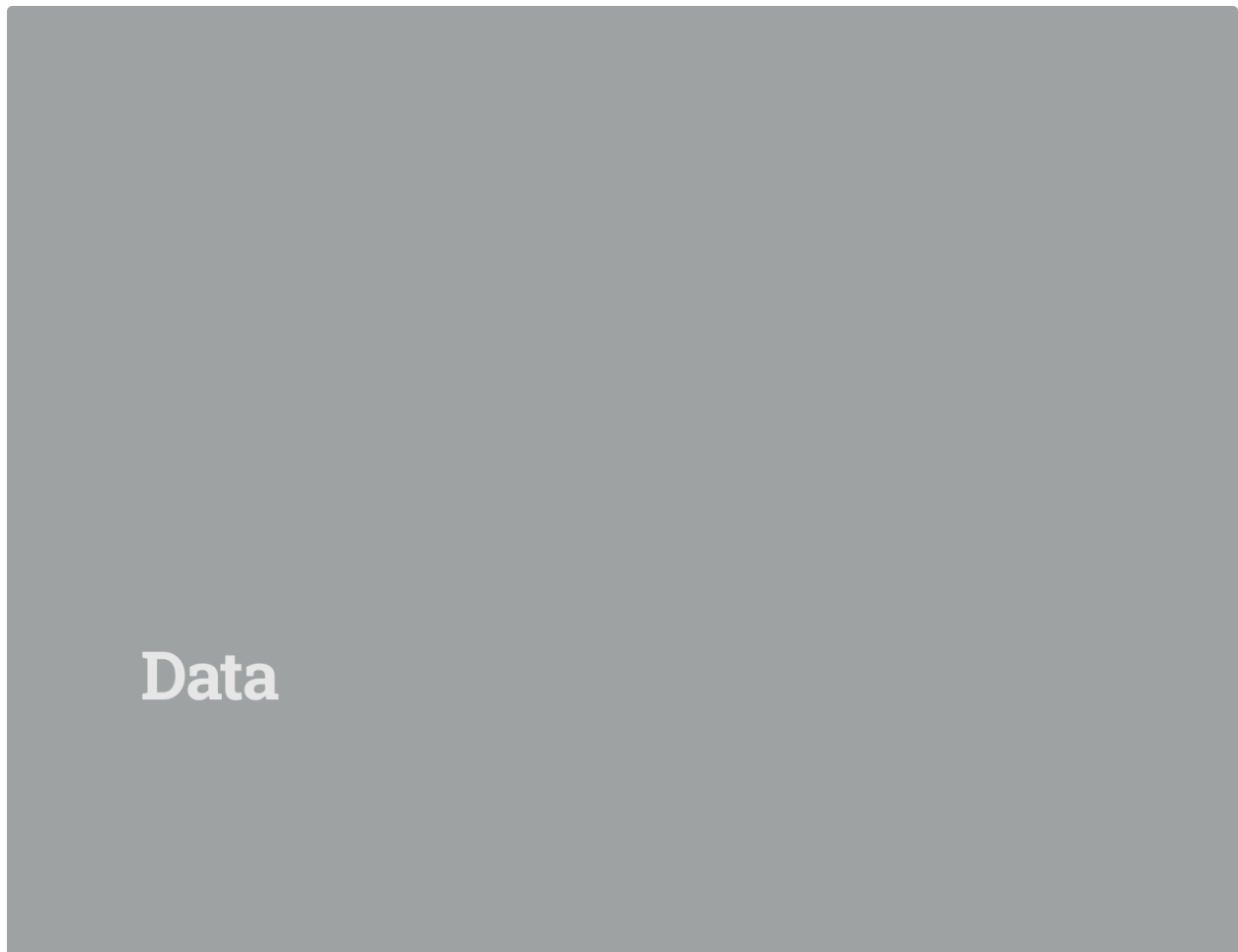


Figure 5.13: Slide 5: Data (section divider)

March 2024 CPS

Insert your content here replacing these words with your own text.

6/35

Figure 5.14: Slide 6: March 2024 CPS

March 2024 CPS Extract

```
cpsmar_e <- ____ (here("____", "____"))
```

Insert your content here replacing these words with your own text.

7/35

Figure 5.15: Slide 7: March 2024 CPS Extract

Analysis sample

```
cpsmar_a <- cpsmar_e %>%
  filter(
    age >= ____,
    age <= ____,
    earnings > ____
  ) %>%
  mutate(
    gender = __(female == ____, "Female", "Male"),
    wage = ____/(____ * ____),
    lwage = log(____),
    Black = case_when(race==____~1, TRUE ~ 0),
    south = case_when(region==____~1, TRUE ~ 0),
    married = case_when((marital==____ | marital==____ | marital==____)~1, TRUE ~ 0),
    age_centered = age - 23
  )
```

Insert your content here replacing these words with your own text.

8/35

Figure 5.16: Slide 8: Analysis sample

5.6.4 Baseline earnings distributions (slides 9–12)



Figure 5.17: Slide 9: Baseline earnings distributions (section divider)

Plotting earnings distributions

```
figure1 <- ggplot(____, aes(x = _____, group = _____, fill = _____)) +
  geom_density(alpha = 0.4) +
  labs(
    title="Figure 1. Distribution of earnings by gender",
    x="Earnings",
    y="Density"
  )+
  theme_minimal()
earnings_fvm <- cpsmar_a %>%
  group_by(____) %>%
  summarize(avg_earnings = round(____(____, na.rm = TRUE),0))

avg_earnings_f <- earnings_fvm %>%
  ____ (gender == "Female") %>%
  pull(avg_earnings) # 'pull' extracts the "avg_earnings" value for "Female" from earnings_fvm, a single value since the data
only record two genders.

avg_earnings_m <- earnings_fvm %>%
  ____ (gender == "Male") %>%
  pull(avg_earnings) # 'pull' extracts the "avg_earnings" value for "Male" from earnings_fvm, a single value since the data only
record two genders.
```

10/35

Figure 5.18: Slide 10: Plotting earnings distributions

Distribution of earnings by gender

11/35

Figure 5.19: Slide 11: Distribution of earnings by gender (Figure 1)

Baseline comparisons

Insert your content here replacing these words with your own text.

12/35

Figure 5.20: Slide 12: Baseline comparisons

5.6.5 The career gender gap (slides 13–20)



Figure 5.21: Slide 13: The career gender gap (section divider)

Wages and hours differences

14/35

Figure 5.22: Slide 14: Wages and hours differences (Table 1)

Documenting the differences

Insert your content here replacing these words with your own text.

15/35

Figure 5.23: Slide 15: Documenting the differences

Plotting career log wage profiles

```

cef_fvm_w <- cpsmar_a %>%
  summarise(
    avg_lwage = mean(cef_fvm_w, na.rm = TRUE))

figure2 <- ggplot(cef_fvm_w, aes(x = Year, y = avg_lwage, color = gender, linetype = gender, linewidth = gender)) +
  geom_point() +
  geom_line() +
  scale_linetype_manual(values = c("Female" = "longdash", "Male" = "solid")) +
  scale_linewidth_manual(values = c("Female" = 0.7, "Male" = 0.5)) +
  guides(linewidth = "none") +
  labs(
    title="Figure 2. Career log-wage profiles for women and men",
    x="Year",
    y="Average log wage"
  ) +
  theme_minimal()

```

16/35

Figure 5.24: Slide 16: Plotting career log wage profiles

Career log wage profiles

17/35

Figure 5.25: Slide 17: Career log wage profiles (Figure 2)

Estimating wage differences over a career

```

males <- cef_fvm_w %>%
  (gender == "Male") %>%
  (avg_lwage_male = avg_lwage) %>%
  (-gender)
females <- cef_fvm_w %>%
  (gender == "Female") %>%
  (avg_lwage_female = avg_lwage) %>%
  (-gender)

diff_fvm <- inner_join(males, females, by = "age_centered") %>%
  (age_centered <= 30) %>%
  (
    diff = avg_lwage_male - avg_lwage_female,
    age_group = cut(
      age_centered,
      breaks = c(-1, 10, 20, 30),
      labels = c("1-10", "11-20", "21-30"))
  ) %>%
  group_by(_____) %>%
  (mean_diff = mean(_____) * 100)

table2 <- kable(
  diff_fvm,
  digits = 2,
  col.names = c("Year Range", "Avg Pct Difference"),
  align = "cc",
  caption = "Table 2. Percent wage differences, first 30 years",
  ) %>%
  kable_styling(position = "center")

```

18/35

Figure 5.26: Slide 18: Estimating wage differences over a career

Evolution of the gender wage gap

19/35

Figure 5.27: Slide 19: Evolution of the gender wage gap (Table 2)

Discussing the gender wage gap evolution

Insert your content here replacing these words with your own text.

20/35

Figure 5.28: Slide 20: Discussing the gender wage gap evolution

5.6.6 Explaining the gender wage gap (slides 21–30)



Figure 5.29: Slide 21: Explaining the gender wage gap (section divider)

Fitting the log wage profiles

```
formula <- ____ ~ ____ + I(____^2)
figure3 <- figure2 +
  geom_smooth(
    method = ____,
    formula = ____,
    aes(group = ____),
    se = FALSE
  ) +
  stat_poly_eq(
    aes(label = after_stat(eq.label)),
    formula = ____,
    parse = TRUE
  ) +
  labs(
    title="Figure 3. Career log-wage profiles with quadratic fits",
    x="Year",
    y="Average log wage"
  ) +
  theme_minimal()+
  theme(legend.position = "bottom")
```

22/35

Figure 5.30: Slide 22: Fitting the log wage profiles

Log wage profiles with quadratic fits

23/35

Figure 5.31: Slide 23: Log wage profiles with quadratic fits (Figure 3)

Gender differences in education

24/35

Figure 5.32: Slide 24: Gender differences in education (Table 3)

Gender differences in demographics

25/35

Figure 5.33: Slide 25: Gender differences in demographics (Table 4)

Documenting differences in characteristics

Insert your content here replacing these words with your own text.

26/35

Figure 5.34: Slide 26: Documenting differences in characteristics

Controlling for education and demographic characteristics

```
singles <- cpsmar_a %>%
  filter(
    married==0,
    child_u6==0
  )
models <- list(
  "Baseline" = lm(
    ~ age_centered + I(age_centered^2),
    data=singles
  ),
  "Add Education" = lm(
    ~ age_centered + I(age_centered^2) + education + education^2 + education^3,
    data=singles
  ),
  "Add Person" = lm(
    ~ age_centered + I(age_centered^2) + education + education^2 + education^3 +
      person + person^2 + person^3,
    data=singles
  ),
  "Add Household" = lm(
    ~ age_centered + I(age_centered^2) + education + education^2 + education^3 +
      person + person^2 + person^3 + household + household^2 + household^3,
    data=singles
  ),
  "Only Singles" = lm(
    ~ age_centered + I(age_centered^2) + education + education^2 + education^3 +
      person + person^2 + person^3 + household + household^2 + household^3,
    data=singles
  )
)
```

27/35

Figure 5.35: Slide 27: Controlling for education and demographic characteristics

Explaining the gender wage gap

29/35

Figure 5.37: Slide 29: Explaining the gender wage gap (Table 5)

Documenting the findings

Insert your content here replacing these words with your own text.

30/35

Figure 5.38: Slide 30: Documenting the findings

5.6.7 Conclusion (slides 31–32)



Figure 5.39: Slide 31: Conclusion (section divider)

Summary

Insert your content here replacing these words with your own text.

32/35

Figure 5.40: Slide 32: Summary

5.6.8 Appendix (slides 33–35)



Figure 5.41: Slide 33: Appendix (section divider)

Data documentation

```
# Define the variables and their descriptions
variables <- data.frame(
  Variable = c(
    "age",
    "earnings",
    "hours",
    "race",
    "marital",
    "HSGrad",
    "SomeColl",
    "CollDeg",
    "region",
    "female",
    "hisp",
    "fulltime"
  ),
  Definition = c(
    "years; capped at 85",
    "____",
    "____",
    "respondent's race (1 = White only, 2 = Black only, 3 = AI only, 4 = Asian only, 5 = Hawaiian/Pacific Islander only (HP), 6 = White-Black, 7 = White-AI, 8 = White-Asian, 9 = White-HP, 11 = Black-Asian, 12 = Black-HP, 13 = AI-Asian, 14 = AI-HP, 15 = Asian-HP, 16 = White-Black-AI, 17 = White-Black-Asian, 18 = White-Black-HP, 19 = White-AI-Asian, 20 = White-AI-HP, 21 = White-Asian-HP, 22 = Black-AI-Asian, 23 = White-Black-AI-Asian, 24 = White-AI-Asian-HP, 25 = White-Black-AI-Asian-HP, 26 = Other 3 race comb., 27 = Other 4 or 5 race comb.)",
    "marital status (1 = Married civilian, 2 = Married AF, 3 = Married absent, 4 = Widowed, 5 = Divorced, 6 = Separated, 7 = Never married)",
    "= 1, if ____",
    "= 1, if ____",
    "= 1, if ____",
    "household region (1 = Northeast, 2 = Midwest, 3 = South, 4 = West)",
    "= 1 if ____",
    "= 1 if Hispanic, Spanish, or Latino",
    "= 1 if ____"
  )
)
```

34/35

Figure 5.42: Slide 34: Data documentation

List of main variables with definitions

This is a list of the main variables used in this project with their definitions.

35/35

Figure 5.43: Slide 35: List of main variables with definitions

5.7 If your output doesn't match the reference

Most formatting mismatches come from a handful of recurring mistakes. See the Common Errors chapter for the diagnosis and fix for each — including:

- CSS in the wrong folder
- Wrong orientation (portrait instead of landscape)
- Knit-to-PDF instead of HTML → PDF
- YAML edits that break the rendering
- Background graphics not enabled in the print dialog

If you've checked all of those and still see a difference, email the teaching team. Attach **both** your `.Rmd` file and the `.pdf` you generated (the one with the wrong output), and include the print dialog settings you used.

6Common Errors and FAQ

If something is broken, look here first.

6.1 Outline (to be written)

- The “did setup work?” diagnostic (`check_setup.R`)
- Common error catalog:
 - CSS in the wrong folder
 - `eval=FALSE` left on a completed chunk
 - YAML edited
 - Wrong working directory
 - Missing packages
 - Knit-to-PDF instead of HTML → PDF
 - Incorrect filename on Gradescope
- Communication policy (no content review by email; one thread per issue; no same-issue cross-emails)
- How to ask for help productively

Note

This chapter is a stub. Content will be filled in as part of Wave 2 of the project rebuild.

7R and AI Resources

A curated list of resources for learning and using R, and for working with AI as part of your data-science practice. Nothing here is required reading — but each item below has earned its place. Skim it now, bookmark it, and return as you go.

7.1 Resources for learning and using R

7.1.1 IDEs for R

RStudio remains the most widely used IDE for R, and, like R, it is open-source. Its “Help” tab is a gateway to a wide range of R and RStudio resources. However, Posit (the company behind RStudio) has released Positron, a next-generation IDE built for data science with both R and Python. Positron combines the familiar feel of RStudio with the extensibility of VS Code, and includes built-in AI assistance via Positron Assistant. If you’re comfortable with RStudio, stick with it; if you’re curious about a more modern interface or plan to work across R and Python, Positron is worth exploring.

7.1.2 Learning R

Norm Matloff, a computer scientist and statistician at UC-Davis, has a nice, fast introduction to (base) R. He also has interesting takes on base R vs the Tidyverse and the R vs Python debate in data science. For a more in-depth treatment, consult the widely used *R for Data Science* by Wickham and Grolemund. They will guide you through the Tidyverse. Tidy Data Tutor helps you visualize your data analysis pipelines. James Scott has a great introduction to R that more clearly maps onto BUSN 5000.

You will find a list of econometrics packages for R at the Comprehensive R Archive Network (CRAN). You might prefer the causal inference task view or machine learning task view.

Learning R Markdown or Quarto should go hand-in-hand with learning R. For R Markdown, get started here and keep a cheatsheet handy. You can get started with Quarto here.

7.1.3 Workflow

At least as important as learning R is understanding basic workflow principles. R-bloggers has a beginner’s guide using RStudio Projects. You can find some of the same principles in chapter 2 of Kieran Healy’s *Data Visualization* book (also linked on the course schedule). Matt Gentzkow and Jesse Shapiro provide a fuller take on workflow principles from the perspective of academic social scientists (also posted on eLC). They even provide a manual for doing as they do.

For a perspective specifically on coding practices in data science education, Pruijm, Gîrjău, and Horton (2023) make the case in “Fostering Better Coding Practices for Data Scientists” (*Harvard Data Science Review*) that good coding habits matter even for beginners. Their argument: it’s far easier to learn good practices alongside R than to unlearn bad habits later. They organize their guidance into a practical top-10 list. Worth reading early in your data-science career.

Suffice it to say that Healy’s book is a great introduction to the wonders of ggplot, which is not R, but indispensable to analysis done in R.

7.2 AI resources

7.2.1 Working with AI

My take is that you should view AI chatbots, like ChatGPT, Claude, and Gemini, as exceedingly productive collaborators with whom you should learn to work, much like you would with a human. Unfortunately, there are no secret prompting strategies or special step-by-step manuals (also like with a human). For some practical guidance along these lines, I recommend that you follow Ethan Mollick’s Substack. If you want to swim in the deeper end of the pool, check out the Substack, Don’t Worry About the Vase.

7.2.2 AI for coding

One place gen AI really helps is in coding. You can get coding help from chat interfaces, but you may prefer a more integrated experience. Several options exist:

- Positron Assistant: If you use Positron, this built-in AI assistant understands your R environment, loaded data, and console history. It integrates GitHub Copilot for code completions and Claude for chat and agent mode.
- GitHub Copilot: Works in RStudio, VS Code, and other IDEs. Here is how to get free access as a student. Here are instructions for RStudio integration.
- Claude Code: A command-line tool that can read and modify files in your project directory, run commands, and understand your codebase context. Simon Couch has written useful posts on using Claude Code for R package development.
- Cursor: An AI-first code editor built on VS Code.

Alternatively, you may find you can get by just vibe coding with a chatbot — describing what you want and iterating on the results.

7.2.3 Learning more about AI

To learn more about AI, including a comparison of different models, check out Generative AI for Beginners, a free course offered by Microsoft Cloud Advocates. OpenAI and Anthropic have their own AI academies when you want to step it up.